Critical Reproduction of a Phishing Website Detection Tutorial

Data Science in Cybersecurity
Layan Awawde
July 2026

Executive Summary

This project critically evaluates Nicolas Papernot's tutorial on phishing website detection with a decision tree. The tutorial claims approximately 90.5% accuracy. The current repository data contain 11,055 rows and 30 predictors, although the README states 2,456 websites. The original code trains on the first 2,000 rows, so with the current file it evaluates on 9,055 rows rather than 456. Re-running that logic gives an accuracy of 0.905, which supports the claim only approximately and exposes a major documentation inconsistency. A more rigorous stratified holdout and cross-validation comparison was performed using Logistic Regression, a controlled Decision Tree, and Random Forest. The analysis also found 5,206 exact duplicate rows, creating a risk of optimistic random-split estimates. The main conclusion is that the tutorial is useful as an introductory demonstration, but its evidence is insufficient for a production cybersecurity claim because it relies on one fixed row-order split, reports only accuracy, lacks uncertainty estimates, and provides no temporal or external validation.

1. Summary of the Source

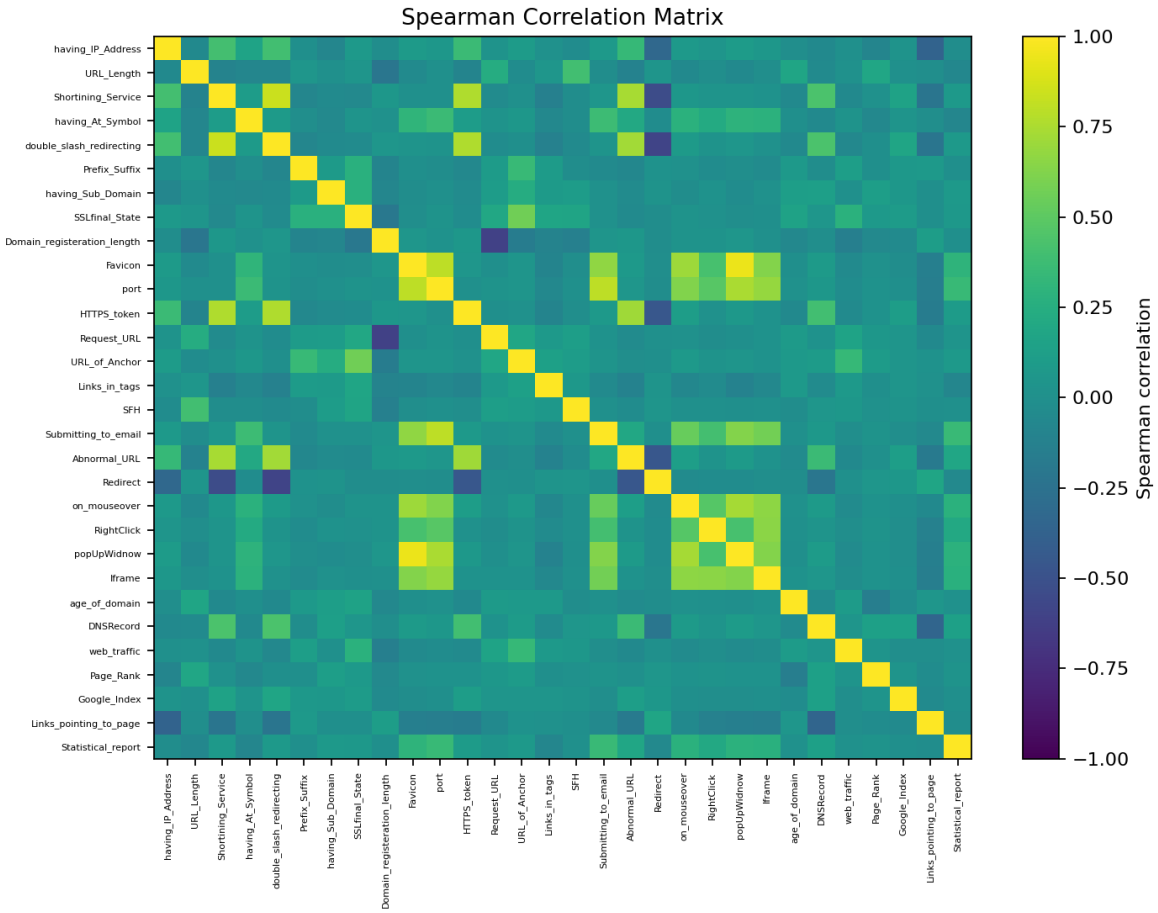
The source addresses binary classification of phishing and legitimate websites. It proposes a scikit-learn DecisionTreeClassifier trained on 30 handcrafted website indicators such as URL length, IP-address use, SSL state, anchor behavior, domain age, web traffic, and statistical reports. The source claims about 90.5% accuracy and also includes a Logistic Regression alternative.

2. Critical Evaluation

The central accuracy claim is broadly reproducible in magnitude: the current data and original-style split produce 0.9051. However, the supporting methodology is weak. The README states 2,456 observations and 456 test cases, whereas the CSV contains 11,055 observations. The code slices the first 2,000 rows for training and all remaining rows for testing, so it currently evaluates 9,055 rows. There is no shuffling, stratification, fixed random seed in the original script, cross-validation, confidence interval, or analysis of class-specific errors. Accuracy alone cannot show whether phishing pages are being missed. Therefore, the source's numerical statement is plausible, but the conclusion should be restricted to this dataset and this implementation rather than presented as evidence of general deployment readiness.

3. Feature Engineering Analysis

The source uses 30 pre-engineered indicators encoded as -1, 0, or 1. These are not raw URLs, so substantial hidden feature extraction has already occurred upstream. No scaling, transformation, aggregation, or feature selection is performed by the tutorial. Scaling is unnecessary for trees but is included for Logistic Regression in this reproduction. Spearman correlation is appropriate because the predictors are ordinal/discrete. Strongly correlated pairs indicate redundancy, which can split feature importance and reduce explainability. Additional useful features could include character n-grams, domain lexical entropy, certificate age, registrar reputation, DNS history, redirect-chain depth, hosting-AS information, visual similarity to known brands, and time-aware campaign features. Their acquisition cost and privacy implications must be documented.



4. Reproducibility Analysis

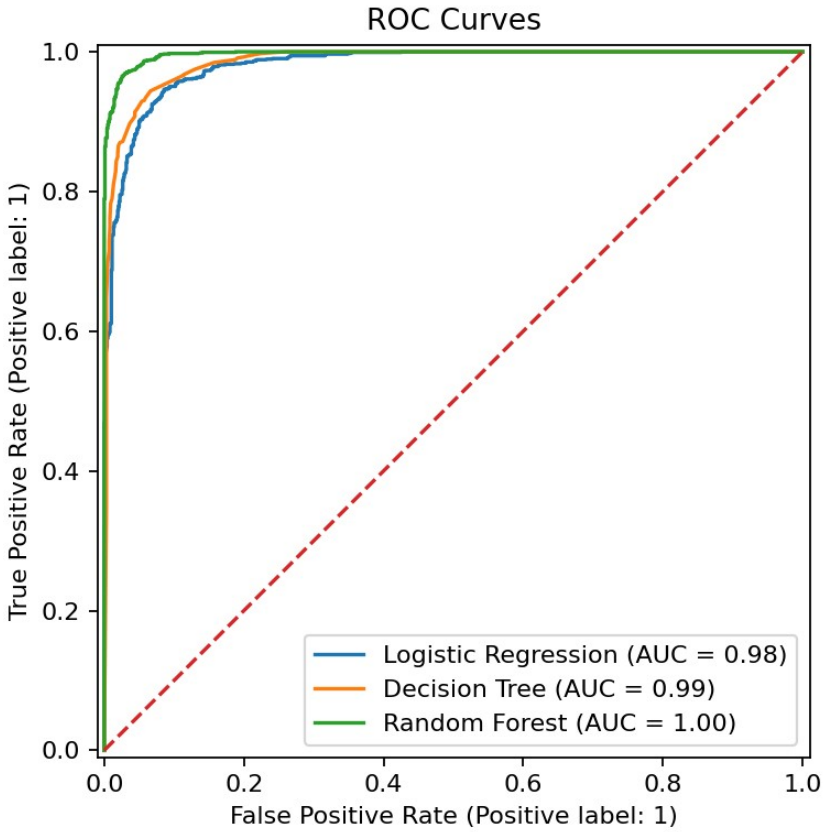
The repository includes code, feature descriptions, and data, which is a strength. Nevertheless, the original script uses Python 2 print syntax and current scikit-learn environments require updates. Dependencies are not pinned. The README and data size disagree. The source also omits the raw website collection process and the code that transforms websites into the 30 indicators. These are hidden preprocessing steps. Reproducibility is therefore partial: the supplied table can be modeled, but the full data-generation pipeline cannot be independently reconstructed from the repository alone.

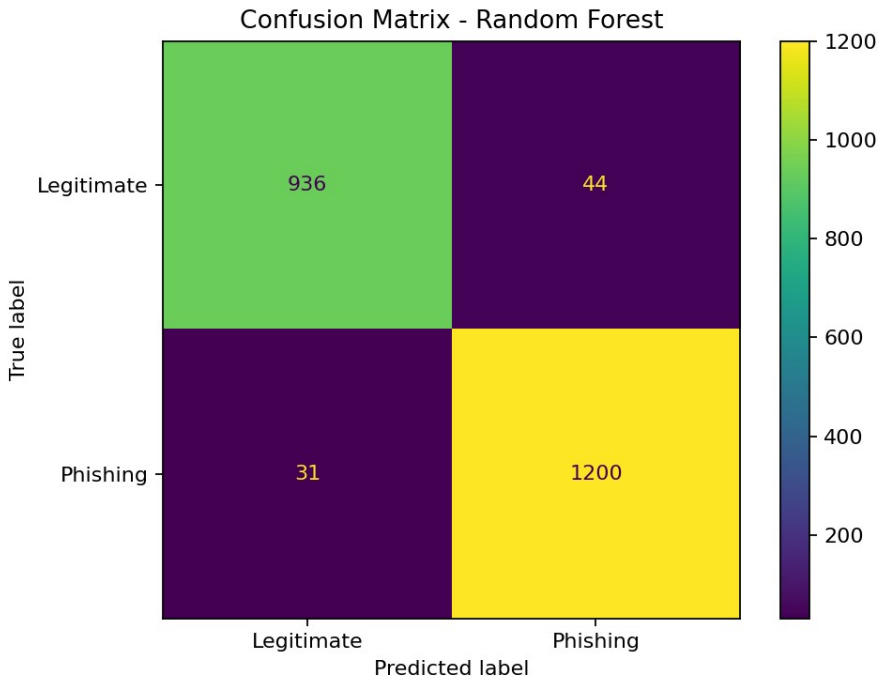
5. Experimental Results

Table 1 reports the stratified 80/20 holdout results. Class 1 is phishing and class -1 is legitimate.

Model	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC
Random Forest	0.9661	0.9646	0.9748	0.9697	0.9728	0.9312	0.9966
Decision Tree	0.9367	0.9557	0.9293	0.9423	0.9345	0.8726	0.9856
Logistic Regression	0.9285	0.9234	0.9504	0.9367	0.9449	0.8551	0.9808

Random Forest achieved the strongest holdout F1 in this run. Cross-validation was also used to estimate average performance and variability rather than relying on one split.





6. Evaluation Metrics and Error Analysis

Accuracy = (TP+TN)/(TP+TN+FP+FN). Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 is the harmonic mean of precision and recall. F2 weights recall more heavily. MCC = (TP*TN-FP*FN)/sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN)). ROC-AUC summarizes ranking quality across thresholds. In phishing detection, a false positive blocks a legitimate site and can create user friction or analyst fatigue. A false negative allows a phishing site to appear legitimate and may expose credentials or payment data. Consequently, recall, F2, MCC, and the confusion matrix are more informative than accuracy alone.

7. Duplicate and Redundancy Analysis

The dataset contains 5,206 exact duplicate rows and 5,270 duplicated feature vectors. Identical examples may appear in both train and test sets under random splitting, causing optimistic estimates. After exact-row deduplication, 5,849 rows remain. Deduplicated evaluation is therefore reported separately and should be treated as more conservative.

Model	Accuracy	Recall	F1	MCC
Random Forest	0.9470	0.9488	0.9454	0.8939
Decision Tree	0.9325	0.9311	0.9303	0.8648
Logistic Regression	0.9128	0.9223	0.9110	0.8258

8. Conclusions

The tutorial succeeds as a compact educational example but does not provide sufficient evidence for robust real-world phishing detection. Its reported accuracy is approximately reproducible, yet the repository is internally inconsistent and the evaluation omits duplicate sensitivity, cross-validation, multiple metrics, and temporal generalization. The strongest improvement is not merely selecting a more powerful model; it is correcting the evaluation design. Future work should use time-stamped and recently collected URLs, group-aware or campaign-aware splits, external test data, calibration, threshold tuning based on incident cost, and transparent feature-extraction code. I would not recommend deploying the tutorial model unchanged on similar operational problems, but I would recommend it as a starting point for a carefully redesigned experiment.

References

Papernot, N. Detecting phishing websites using a decision tree. GitHub repository: https://github.com/npapernot/phishing-detection

UCI Machine Learning Repository. Phishing Websites dataset.